NED UNIVERSITY OF ENGINEERING & TECHNOLOGY

Department of Computer and Information Systems Engineering

Software Engineering • Course Project

SOFTWARE REQUIREMENTS SPECIFICATION

AI-Powered Co-Working Space Asset Manager

An intelligent web system for tracking, requesting and maintaining shared assets

DOCUMENT INFORMATION

Document	SRS
Version	1.0
Status	Submitted for review
Date	October 2026
Standard	IEEE Std 830-1998

PREPARED BY

Hooria Ahmed	CS-24106
Misbah Ul Hasan	CS-24130
Syed Azlaan Aqib	CS-24144
M. Hamza Siddiqui	CS-24139

Submitted to

Ms. Fakhra Aftab

Document Control

Revision History

Version	Date	Description of Change	Prepared by
1.0	October 2026	First complete version of the SRS, submitted for review.	All group members

Review and Approval

Role	Name	Signature	Date
Reviewed by	Practical Class Teacher		
Approved by	Theory Class Teacher		

Contents

1 Introduction 1

1.1 Document Purpose . . . 1
1.2 Product Scope . . . 1
1.3 Intended Audience and Document Overview . . . 2
1.4 Definitions, Acronyms and Abbreviations . . . 3
1.5 Document Conventions . . . 3
1.6 References and Acknowledgments . . . 4

2 Overall Description 5

2.1 Product Perspective . . . 5
2.2 Product Functionality . . . 6
2.3 Users and Characteristics . . . 7
2.4 Operating Environment . . . 8
2.5 Design and Implementation Constraints . . . 10
2.6 User Documentation . . . 10
2.7 Assumptions and Dependencies . . . 11

3 Specific Requirements 13

3.1 External Interface Requirements . . . 13
3.2 Functional Requirements . . . 15
3.3 Behaviour Requirements . . . 20

4 Other Non-Functional Requirements 23

4.1 Performance Requirements . . . 23
4.2 Safety and Security Requirements . . . 23
4.3 Software Quality Attributes . . . 25

5 Other Requirements 30

5.1 Database Requirements . . . 30
5.2 Legal and Privacy Requirements . . . 30
5.3 Internationalization Requirements . . . 31
5.4 Reuse Objectives . . . 31
5.5 Future Enhancements . . . 31

A Data Dictionary 32

A.1 Constants . . . 32
A.2 State Variables . . . 33
A.3 Inputs . . . 33
A.4 Outputs . . . 34
A.5 Database Tables . . . 35

B Requirements Traceability Matrix 36

List of Figures

Figure 2.1 System overview of CWAM 5
Figure 2.2 Three-tier architecture of CWAM. 6
Figure 2.3 Data Flow Diagram Level 0 (context diagram) 6
Figure 2.4 Data Flow Diagram Level 1 7
Figure 2.5 Deployment view of the operating environment 9

Figure 3.1 User interface layouts of the main screens 14
Figure 3.2 Use case diagram of CWAM 21
Figure 3.3 State diagram of an asset 22
Figure 3.4 State diagram of an asset request 22

CHAPTER 1

Introduction

The AI-Powered Co-Working Space Asset Manager (CWAM) is a web application that helps a co-working space keep track of its shared equipment. It keeps a central record of every asset, lets members find and request equipment, lets administrators approve requests and track borrowing, returns and maintenance, and adds two AI features: a natural-language *AI Asset Assistant* and an *AI Asset Recommendation* engine. This chapter gives the purpose and scope of the product, its intended audience, the terms used in this document and the references it is based on.

1.1 Document Purpose

This document describes the software requirements of the AI-Powered Co-Working Space Asset Manager (CWAM), release Version 1.0. It:

- Specifies the functional features of the system (authentication, asset management, asset search, requests, borrowing and returns, maintenance, usage history, AI Asset Assistant, AI Asset Recommendation, dashboard and reports, and notifications), with the interface details, design constraints and non-functional requirements.
- Covers the complete system, not a single subsystem: the React.js web client, the Node.js and Express server, the MySQL database and the interface to the external AI service.
- Serves as the agreed basis for the design, implementation and testing of the system, and as the reference against which the finished product will be evaluated.

1.2 Product Scope

Co-working spaces share many assets among their members: laptops, desktop computers, projectors, lab equipment, cables, adapters and other resources. These are often tracked in registers or spreadsheets, which leads to lost items, double bookings, unknown availability and poor maintenance records.

CWAM replaces this manual process with a single web-based system. It will:

- Maintain a central record of all assets with their category, location, condition, availability status and usage history.
- Allow members to search for available assets and submit requests for a chosen period.
- Allow administrators to add and manage assets, approve or reject requests, record the issue and return of assets, and log maintenance activities.
- Provide an **AI Asset Assistant** through which users ask questions in plain language (for example, “Is any projector free tomorrow afternoon?”) and receive answers based on live system data.
- Provide an **AI Asset Recommendation** feature that suggests the most suitable available assets based on the user’s requirement, purpose and requested duration.
- Provide dashboards and reports on asset utilization, overdue items and maintenance.

Benefits. The main benefits of the product are:

- **Less manual work:** requests, approvals, issue and return are recorded once, in one place.
- **Better utilization:** members see what is free, so idle equipment is used and double bookings end.

- **Accountability:** every asset has a history of who used it, when, and in what condition it came back.
- **Fewer breakdowns:** damaged equipment is taken out of circulation until it is repaired.
- **Faster help:** the AI features answer questions and suggest suitable assets in seconds.
- **Informed decisions:** reports on usage, overdue items and maintenance costs show what to buy or retire.

> **Out of scope for Version 1.0.** Payment and billing, desk or room booking, procurement of new assets, and support for more than one co-working site (see Chapter 5 for planned enhancements).

1.3 Intended Audience and Document Overview

This document is intended for:

- **Course instructors**, who act as the client and evaluators of the project.
- **The development team**, who will design and build the system.
- **Testers**, who will verify it against these requirements.
- **Documentation writers**, who will prepare the user manual, and **future maintainers**.

The rest of this SRS is organized as follows:

- **Chapter 2**: overall description of the product, with the system overview, architecture, data flow and deployment diagrams.
- **Chapter 3**: interface, functional and behaviour requirements, with screen layouts, use case and state diagrams.
- **Chapter 4**: performance, safety, security and software quality attributes.
- **Chapter 5**: database, legal, internationalization and reuse requirements, and future enhancements.
- **Appendix A**: data dictionary of constants, state variables, inputs, outputs and database tables.
- **Appendix B**: traceability from use cases to functional requirements.

Table 1.1 suggests a reading sequence for each type of reader. Every reader should begin with Chapter 1 and Sections 2.1 to 2.3.

Table 1.1. Suggested reading sequence

Reader	Suggested sequence after the overview
Instructor / Client	Section 2.4 to 2.7, Section 3.3 (use cases), Chapter 4, then Section 3.2 as needed.
Developers	Sections 2.4 and 2.5, Section 3.1, Section 3.2 in full, Chapter 5 and Appendix A.
Testers	Section 3.2 (requirement IDs), Section 3.3, Chapter 4 and Appendix B.
Documentation writers	Section 2.6, Section 3.1.1 (screens), Section 3.3 and Appendix A (inputs and outputs).
Maintainers	Section 2.5, Section 4.3 (design for change), Chapter 5 and Appendix A.

1.4 Definitions, Acronyms and Abbreviations

The terms below are sorted alphabetically, reading down the left column and then the right column.

Table 1.2. Definitions, acronyms and abbreviations

Term	Definition	Term	Definition
Administrator (Admin)	Staff member who manages assets, users and requests.	**LTS**	Long-Term Support release of a software product.
AI	Artificial Intelligence.	**Member (User)**	Person using the co-working space who borrows assets.
AI Asset Assistant	Chat feature that answers asset questions written in natural language.	**MySQL**	Open-source relational database management system.
AI Asset Recommendation	Feature that ranks available assets against a user's stated need.	**NFR**	Non-Functional Requirement.
API	Application Programming Interface.	**Node.js**	JavaScript runtime used to build the server.
Asset	Any physical item tracked by the system, e.g. a laptop or projector.	**Overdue**	Borrowing whose due date has passed without a return.
Asset Tag	Unique code, serial number or QR label of an asset.	**PDF**	Portable Document Format.
bcrypt	Password-hashing algorithm producing salted, slow-to-compute hashes.	**QA**	Quality Attribute.
Borrow Record	Record of an asset issued to a user, with issue, due and return dates.	**RBAC**	Role-Based Access Control.
CRUD	Create, Read, Update, Delete.	**React.js**	JavaScript library for building user interfaces.
CSV	Comma-Separated Values, a plain-text table format.	**REST**	Representational State Transfer, the style of the system's API.
CWAM	Co-Working Space Asset Manager (this system).	**SPA**	Single-Page Application.
DFD	Data Flow Diagram.	**SQL**	Structured Query Language.
Express	Web application framework for Node.js.	**SRS**	Software Requirements Specification.
FR	Functional Requirement.	**TLS**	Transport Layer Security, the encryption used by HTTPS.
HTTPS	Hypertext Transfer Protocol Secure.	**UC**	Use Case.
JSON	JavaScript Object Notation.	**UI**	User Interface.
JWT	JSON Web Token, used to authenticate users after login.	**UML**	Unified Modeling Language.
LLM	Large Language Model, used by the AI features.	**XSS**	Cross-Site Scripting, injection of malicious scripts into web pages.

1.5 Document Conventions

In general this document follows the IEEE formatting requirements of IEEE Std 830-1998 and the section and subsection titles of the course SRS template. It has been prepared with LaTeX.

1.5.1 Formatting Conventions

- Body text uses an Arial-style sans-serif font (Helvetica, the LaTeX equivalent of Arial) at 11 pt, with single line spacing and 1-inch margins on all sides.
- *Italics* are used for comments, notes, example inputs and the names of screen elements such as the *Request* button.
- **Bold** text is used for headings, key terms and the names of actors.
- Requirement identifiers are shown in a bold navy font, for example **FR-AM-01**. Tables have a navy header row, and figures and tables are numbered by chapter.
- Each functional module gives a *user requirement* in a shaded box followed by a table of numbered *system requirements*.

1.5.2 Naming Conventions

- Functional requirements are identified as **FR-XX-nn**, where *XX* is the module code (for example *AM* for Asset Management) and *nn* is the requirement number. User requirements use **UR-XX**.
- Non-functional requirements are identified as **NFR-XX-nn**, where *XX* is *PF* (performance), *SF* (safety) or *SC* (security). Software quality attributes use **QA-nn**.
- Other requirements in Chapter 5 are identified as **OR-XX-nn**, where *XX* is *DB* (database), *LG* (legal and privacy), *IN* (internationalization) or *RU* (reuse).
- Use cases are identified as **UC-nn**. Constants in the data dictionary are written in capitals, for example `MAX_BORROW_DAYS`, and database fields in lower case, for example `asset_tag`.
- “Shall” marks a mandatory requirement; “should” marks a desirable one.

1.5.3 Priority Conventions

Every requirement has one of three priorities:

- **High**: essential for the first release.
- **Medium**: important, planned for the release.
- **Low**: desirable, implemented if time permits.

1.6 References and Acknowledgments

References are listed in IEEE citation style.

[1] *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1998, 1998.
[2] I. Sommerville, *Software Engineering*, 10th ed. Boston, MA, USA: Pearson, 2016.
[3] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner’s Approach*, 9th ed. New York, NY, USA: McGraw-Hill Education, 2020.
[4] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” IETF, RFC 7519, May 2015.
[5] OWASP Foundation, “OWASP Top Ten,” [Online]. Available: https://owasp.org/www-project-top-ten/
[6] Meta Open Source, “React documentation,” [Online]. Available: https://react.dev
[7] OpenJS Foundation, “Express documentation,” [Online]. Available: https://expressjs.com
[8] OpenJS Foundation, “Node.js documentation,” [Online]. Available: https://nodejs.org/docs
[9] Oracle Corporation, “MySQL 8.0 Reference Manual,” [Online]. Available: https://dev.mysql.com/doc
[10] H. Ahmed, M. U. Hasan, S. A. Aqib, and M. H. Siddiqui, “AI-Powered Co-Working Space Asset Manager,” Project Proposal, Dept. of Computer and Information Systems Engineering, NED University of Engineering and Technology, Karachi, Pakistan, Sep. 2026.

Acknowledgments. The group thanks the course instructors for their guidance and for the SRS template and sample documents on which the structure of this document is based.

CHAPTER 2

Overall Description

2.1 Product Perspective

CWAM is a new, self-contained web system. It replaces paper registers and spreadsheets currently used to track shared equipment. Figure 2.1 shows the system with its users and the main responsibilities of each.

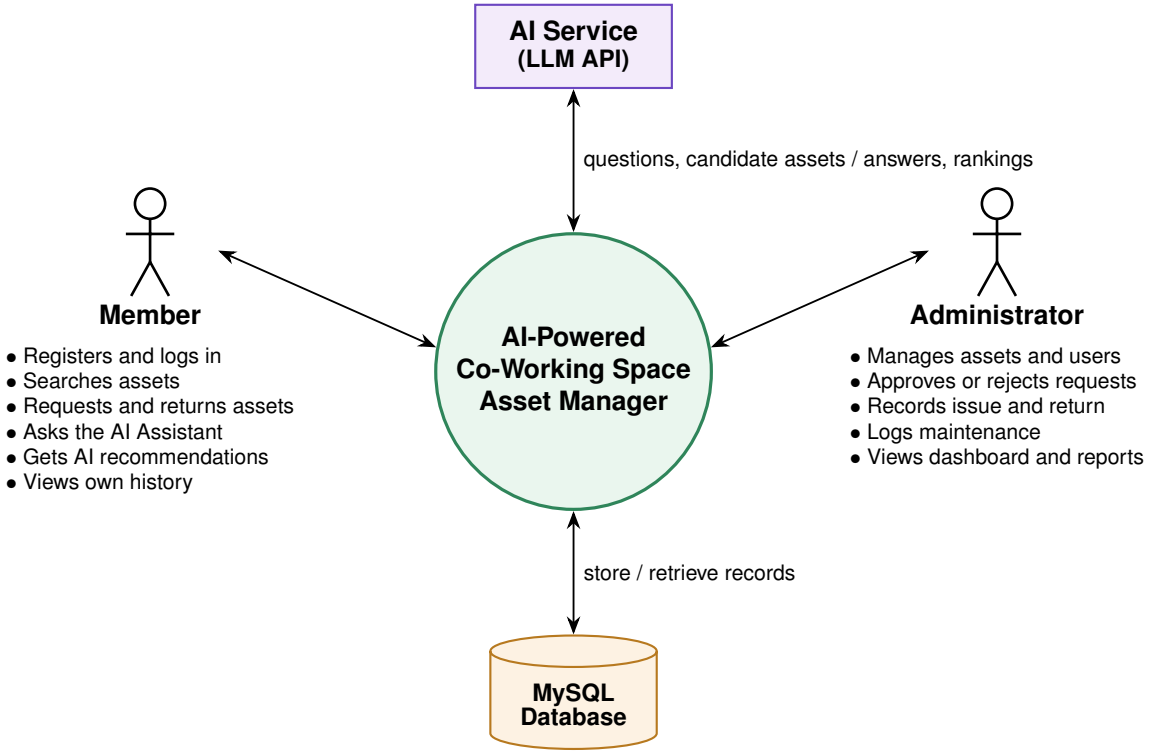


Figure 2.1. System overview of CWAM

The system follows a three-tier client–server architecture, shown in Figure 2.2:

1. **Presentation tier:** a React.js single-page application running in the user's web browser.
2. **Application tier:** a Node.js and Express REST API that holds the business logic, authentication (JWT) and the AI integration.
3. **Data tier:** a MySQL database storing users, assets, requests, borrow records, maintenance logs and AI chat logs.

The server also communicates with an external LLM API:

- The LLM API powers the AI Asset Assistant and AI Asset Recommendation.
- The AI service never accesses the database directly: the server queries the database and passes only the data that is needed to the AI service.

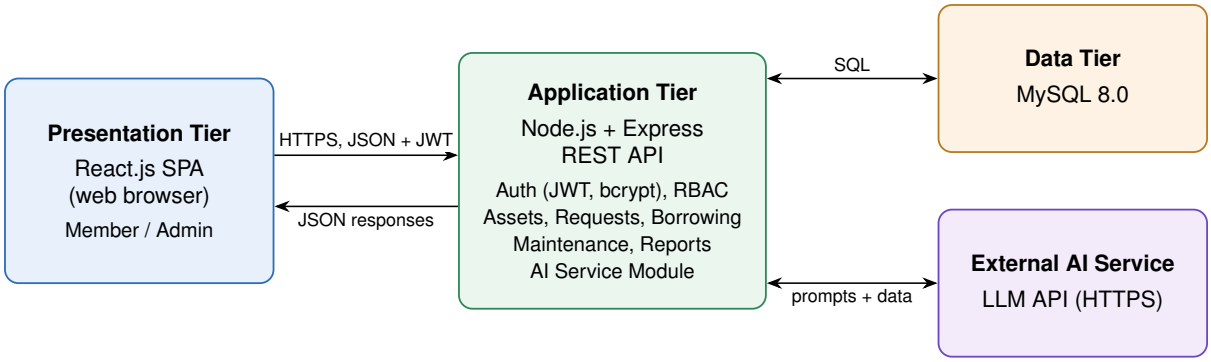


Figure 2.2. Three-tier architecture of CWAM

2.2 Product Functionality

The major functions of CWAM are listed below. The Data Flow Diagrams in Figures 2.3 and 2.4 show how data moves through the system. The system will:

- Allow users to register, log in and log out securely.
- Allow the administrator to add, view, edit and delete assets with their details (tag, name, category, location, condition, status, description and image).
- Allow users to search assets by name and filter them by category, location, status and condition.
- Show the full details and current availability of each asset.
- Allow users to submit a request for an available asset with a purpose, start date and duration.
- Allow the administrator to approve or reject requests, with a reason for rejection.
- Record the issue of an asset to a user and its return, including the condition on return.
- Flag borrowed assets that are not returned by their due date as overdue.
- Allow the administrator to log maintenance issues, mark assets as under maintenance and close maintenance records.
- Keep a usage history for each asset and each user.
- Allow users to ask asset-related questions in natural language through the AI Asset Assistant.
- Recommend suitable available assets through the AI Asset Recommendation feature, based on requirement, purpose and duration.
- Show the administrator a dashboard with asset counts by status, most-used assets, overdue items and a maintenance summary.
- Notify users in the application when their request is approved or rejected and when a return is due.

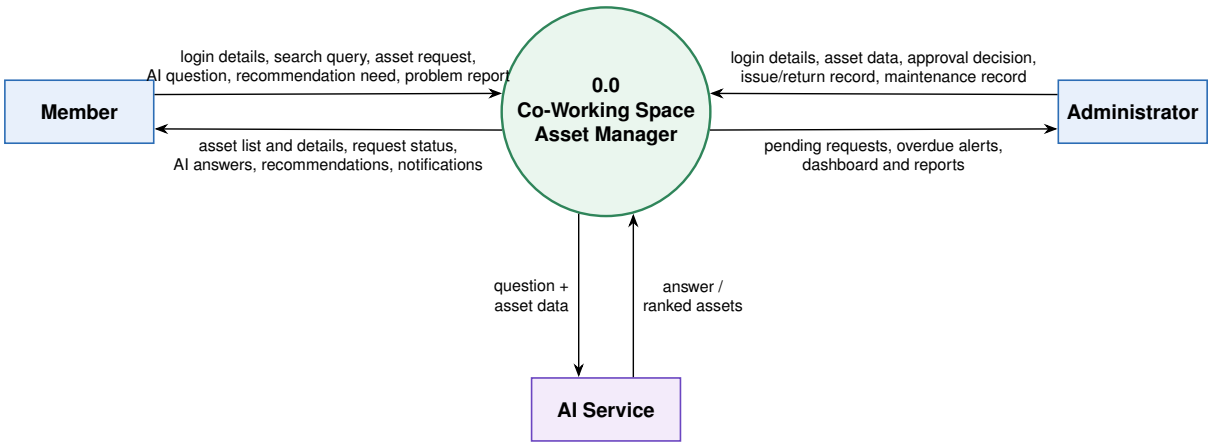


Figure 2.3. Data Flow Diagram Level 0 (context diagram)

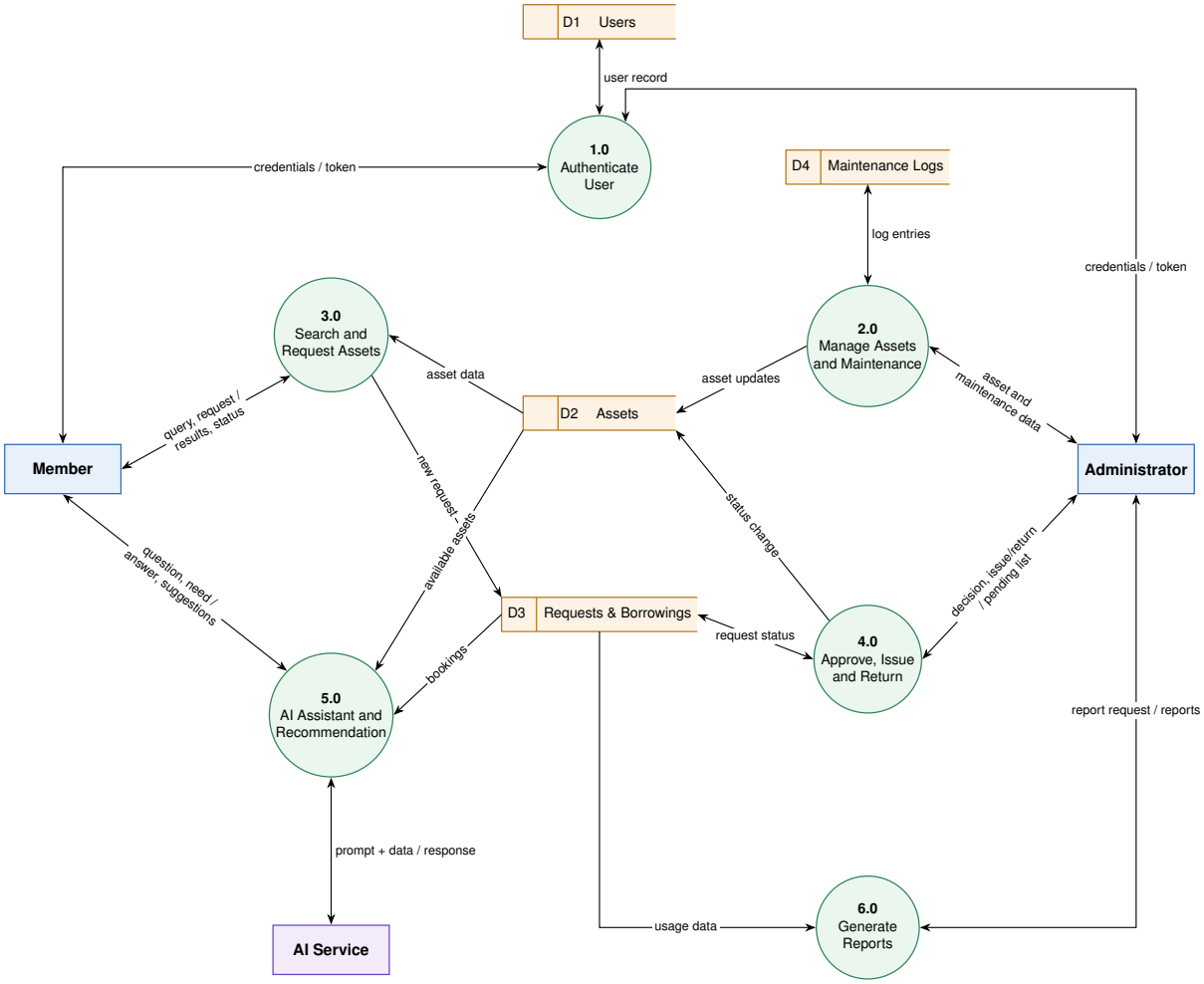


Figure 2.4. Data Flow Diagram Level 1

2.3 Users and Characteristics

Table 2.1. User classes and characteristics

User Class	Responsibilities	Technical Expertise	Usage	Importance
Member (User)	Freelancers, students and employees working in the space. Search for assets, submit requests, view borrowed items and history, and use the AI Assistant and AI Recommendation.	Basic web browsing; no technical knowledge required.	Frequent	**Primary**
Administrator	Staff member with all privileges. Manages the asset inventory, user accounts, requests, the issue and return of assets, maintenance records and reports.	Basic computer skills and good knowledge of the space's equipment.	Daily	**Secondary**

Most important users.

- Members are the *favoured* user class: they are the largest group, and the system only succeeds if they find it easier than asking staff.

- When the needs of the two classes conflict, the member's ease of use is preferred, except where security or the accuracy of asset records would suffer; then the administrator's requirements take priority.
- Administrators are fewer, but the correctness of all data depends on them.

2.3.1 Member (User)

- **Who:** freelancers, students and employees who need equipment from time to time; the large majority of users.
- **Goals:** see what is available, request an asset for a period, track their requests and get quick answers and suggestions from the AI features.
- **Skills and needs:** non-technical and use the system occasionally without training, so they need clear navigation, plain-language messages and confirmation of every action.
- **Access:** the catalogue and only their own requests, borrowings and notifications, usually from their own laptop or phone.

2.3.2 Administrator

- **Who:** staff members of the space (one to five people) responsible for the equipment.
- **Goals:** keep the inventory accurate, process requests promptly, record every issue and return, track maintenance and monitor usage.
- **Skills and needs:** work in the system daily from the front-desk computer, so they need fast lists, filters and clear warnings before irreversible actions.
- **Access:** full access to all functions and data.

2.3.3 Shared Characteristics

- All users need a consistent layout, clear feedback after every action and responsive pages that work from a 360-pixel-wide phone screen up to a desktop monitor.
- All users must log in, and all can use the AI Asset Assistant within the limits of their role.

2.4 Operating Environment

CWAM is a web-based client–server system:

- **Clients:** members and administrators use it through a web browser on their own laptops, phones or the front-desk computer, so nothing has to be installed on client devices.
- **Server:** a single machine (a local computer in the space or a cloud virtual machine) hosts the Node.js and Express application, the built React files, the uploaded asset images and the MySQL database.
- **AI service:** the server reaches the external LLM API over the internet for the AI features.
- **Coexistence:** only ordinary web ports are needed (HTTPS on port 443), so the system coexists with the other software and services on the space's network without conflict.

Figure 2.5 shows the major components, their interconnections and the external interfaces, and Table 2.2 gives the minimum platform requirements.

2.4.1 Client Environment

- The latest two versions of Google Chrome, Mozilla Firefox, Microsoft Edge or Safari, with JavaScript and cookies enabled.
- Windows, macOS or Linux on desktops and laptops; Android or iOS on tablets and phones.
- A responsive layout from 360 pixels wide (phones) up to full desktop screens.

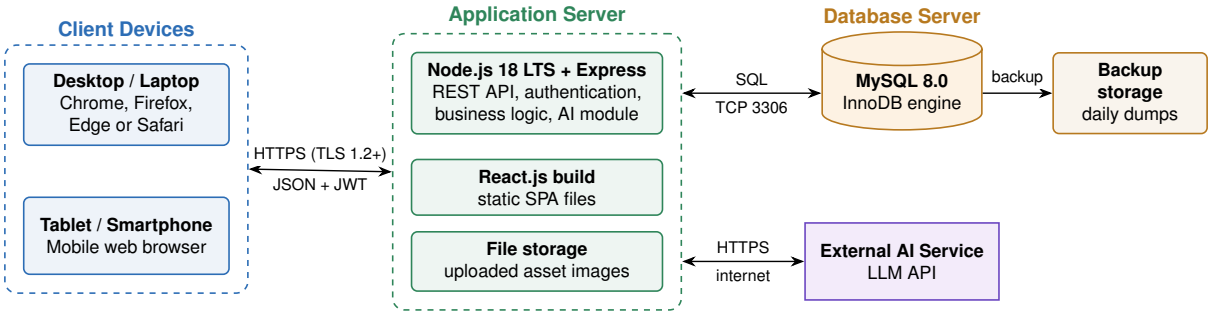


Figure 2.5. Deployment view of the operating environment

2.4.2 Server Environment

- Node.js 18 LTS or later with Express, managed by a process manager (for example PM2) so that it restarts automatically after a failure.
- MySQL 8.0 Community Server with the InnoDB storage engine, on the same machine for Version 1.0.
- Ubuntu 22.04 LTS or Windows 10/11 (Windows Server 2019 or later for a permanent installation).
- A reverse proxy such as Nginx with a TLS certificate to provide HTTPS in deployment.

2.4.3 Network Environment

- The space's local network or the internet between clients and the server, using HTTPS.
- Outbound internet access from the server to the LLM API. Without it, every feature except the AI features continues to work.
- The system should be available 24/7. Short planned interruptions for maintenance can be tolerated.

2.4.4 Minimum Platform Requirements

Table 2.2. Minimum and recommended platform requirements

Item	Minimum	Recommended
CLIENT DEVICE		
Processor	Dual-core, 1.6 GHz	Quad-core, 2.0 GHz or better
Memory (RAM)	4 GB (phones: 2 GB)	8 GB
Operating system	Windows 10, macOS 12, Ubuntu 20.04, Android 10 or iOS 15	Latest version
Browser	Chrome, Firefox, Edge or Safari (latest two versions)	Latest Chrome or Edge
Screen resolution	360 × 640 (phone), 1366 × 768 (desktop)	1920 × 1080
Network	2 Mbps	10 Mbps or faster
SERVER (APPLICATION AND DATABASE ON ONE MACHINE)		
Processor	2 cores (2 vCPU)	4 cores (4 vCPU)
Memory (RAM)	4 GB	8 GB
Storage	20 GB SSD	50 GB SSD
Operating system	Ubuntu 22.04 LTS or Windows 10/11	Ubuntu 22.04 LTS or Windows Server 2019+

Item	Minimum (continued)	Recommended
Software	Node.js 18 LTS, MySQL 8.0	Node.js 20 LTS, MySQL 8.0, Nginx, PM2
Network	10 Mbps, outbound HTTPS to the LLM API	50 Mbps, fixed domain name with TLS certificate

2.5 Design and Implementation Constraints

The constraints in Table 2.3 limit the options available to the developers.

Table 2.3. Design and implementation constraints

No.	Constraint	Description
C-01	**Technology**	The system shall be developed using React.js for the frontend, Node.js with Express for the backend, MySQL for the database and JWT for authentication, as approved in the project proposal.
C-02	**Browser**	A modern web browser with JavaScript enabled is required to use the system.
C-03	**Internet**	The AI features need the server to reach the external LLM API over the internet; without it, all other features continue to work.
C-04	**AI service**	The LLM API may have usage limits, costs and response delays. AI responses must be generated only from data supplied by the system, so that the AI does not invent assets.
C-05	**Security**	Passwords are stored only as salted bcrypt hashes; the AI API key and the JWT secret stay on the server and are never sent to the browser.
C-06	**Time**	The system must be designed, built, tested and documented within one academic semester by a team of four students.
C-07	**Hardware and cost**	Version 1.0 runs on one modest server (Table 2.2) using only free and open-source software. The only recurring cost is the LLM API, so AI requests must be limited (FR-AI-09).
C-08	**Design conventions and programming standards**	Because others will maintain the system after the course, the code shall use one Git repository with separate client and server folders, a layered server (routes, controllers, services, data access), REST endpoint naming (e.g. `GET /api/assets`), ESLint and Prettier, and configuration in a `.env` file that is never committed.
C-09	**Business rules (server-enforced)**	At most one member holds an asset at a time and approved requests never overlap (FR-AP-02); requests last 1 to 30 days and cannot start in the past (FR-RQ-02, FR-RQ-03); asset tags are unique and Reserved or Borrowed assets cannot be deleted (FR-AM-02, FR-AM-06); history cannot be edited (FR-UH-03); the AI can only read data (FR-AI-06).

2.6 User Documentation

Because most members will use the system without any training, the main form of help must be inside the application itself, supported by short manuals for each user class. The following documentation will be delivered with the software:

- **Member Guide** (PDF, about 10 pages, with screenshots): registering, logging in, searching for assets, making and cancelling requests, reading notifications, and using the AI Assistant and AI Recommendation.
- **Administrator Manual** (PDF): managing assets and categories, approving and rejecting requests, issuing assets and recording returns, maintenance, users and reports.

- **In-application help:** tooltips on form fields, example questions in the AI chat panel, empty-state hints and clear error messages that say how to fix the problem.
- **FAQ page** inside the application for common questions such as forgotten passwords and overdue items.
- **Installation and Deployment Guide** (README in the code repository) for the people who install and maintain the server.

The manuals will be written in English and kept in the repository so that they are updated with each release.

2.7 Assumptions and Dependencies

This section lists the assumptions on which the requirements are based and the external factors on which the project depends.

2.7.1 Assumptions

The design is based on the sizing assumptions in Table 2.4. Each number has a direct effect on the design; if the real figures turn out to be much larger, the design decisions in the last column must be revisited.

Table 2.4. Sizing assumptions and their effect on the design

Assumption	Assumed value	Effect on the design
Co-working sites	1 site (several rooms and floors)	One database with no multi-site separation; *location* is a simple text field.
Assets in inventory	Up to 5,000	A single MySQL server with indexes on tag, status and category is enough; lists are paginated.
Registered members	Up to 500	Simple user table; no need for groups or organisations.
Concurrent users	Up to 50	A single Node.js server; no load balancer or caching layer in Version 1.0.
Administrators	1 to 5	One administrator role with full rights; no admin hierarchy.
Requests per day	Up to 200	Approval is done manually by an administrator; no automatic approval.
Borrowing period	1 to 30 days	Dates are stored as calendar days; no hourly booking.
AI questions	Up to 50 per user per day	Daily limit per user controls the cost of the LLM API (FR-AI-09).

Other assumptions:

- Every asset has a unique asset tag or serial number, and administrators enter asset data correctly and keep it up to date.
- Each user has a valid email address and their own account; accounts are not shared.
- Users hand assets over physically at the front desk, where an administrator records the issue and the return in the system.
- All users read English and all times are in a single time zone (Pakistan Standard Time).
- Users access the system through a device with a modern web browser and a working internet or local network connection.

2.7.2 Dependencies

The project depends on the external factors in Table 2.5. The last column states what happens if a dependency fails.

Table 2.5. Dependencies and the effect of their failure

Dependency	Used for	If it fails
Server machine	Hosting the Node.js application and the MySQL database.	The whole system is unavailable until the server is restored; data is recovered from the latest daily backup.
External LLM API	AI Asset Assistant and AI Asset Recommendation.	AI chat shows an error message; recommendations fall back to rule-based ranking (FR-RC-07). All other features keep working.
Open-source libraries	React, Express, a MySQL driver, `jsonwebtoken`, `bcrypt` and a chart library.	Versions are fixed in `package-lock.json`, so updates cannot break the build unexpectedly.
Network connectivity	Communication between clients, server and AI service.	Users cannot reach the system; if only the internet link fails, the system remains usable on the local network without AI features.
Front-desk procedures	Physical hand-over and return of assets.	Records will not match reality; this is controlled through the overdue list and regular stock checks.

CHAPTER 3

Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

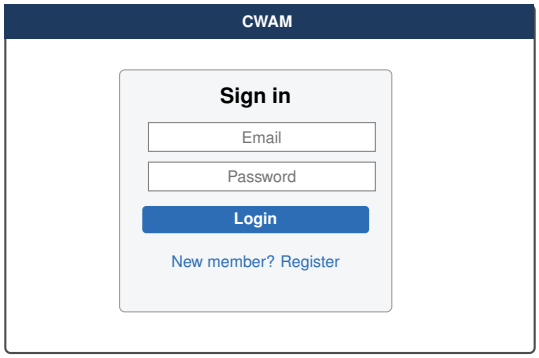
All screens share a consistent layout: a top navigation bar, a side menu that depends on the user's role, and a content area. The interface is responsive. The main screens are listed in Table 3.1, and Figure 3.1 shows the layout of four of them.

Table 3.1. Main screens of the system

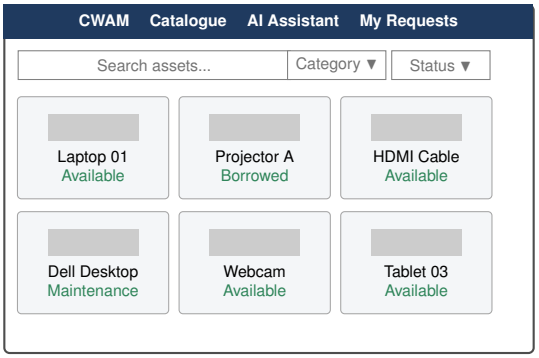
No.	Screen	Contents	User
1	**Login / Register**	Email and password; registration adds name and contact number.	All
2	**Member Dashboard**	Active borrowings, pending requests, due dates and notifications.	Member
3	**Asset Catalogue**	Assets in a grid with a search box and filters for category, location, status and condition.	All
4	**Asset Details**	Full information about one asset, its availability and a *Request* button when available.	All
5	**Request Form**	Purpose, start date and duration.	Member
6	**My Requests and History**	Status of all the member's requests and past borrowings.	Member
7	**AI Asset Assistant**	Chat panel where the user types questions and sees answers.	All
8	**AI Recommendation**	Form for requirement, purpose and duration; ranked suggestions with reasons and a *Request* button for each.	Member
9	**Admin Dashboard**	Asset counts by status, overdue list, recent requests and maintenance summary with charts.	Admin
10	**Management Screens**	Manage Assets, Manage Requests, Issue/Return, Maintenance, Manage Users and Reports.	Admin

The following GUI standards apply to every screen:

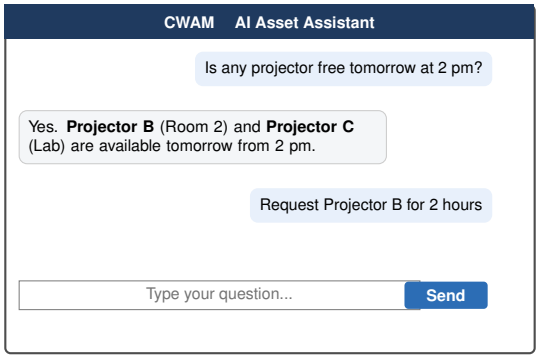
- **Layout:** the same top bar (logo, user name, notifications bell, *Logout*) and a role-based side menu on every page; on phones the side menu collapses into a menu button.
- **Standard buttons:** every form has a primary action button (for example *Save* or *Submit*) and a *Cancel* button that returns to the previous screen without saving. Lists have *View*, *Edit* and *Delete* actions where the user's role allows them.
- **Confirmation:** actions that cannot be undone (delete asset, reject request, deactivate user, retire asset) open a confirmation dialog first.
- **Messages:** validation errors appear in red under the field concerned; success and failure of an action are shown as a short message at the top right that disappears after a few seconds. Messages are in plain language and say how to fix the problem.
- **Status colours:** asset and request statuses are shown as coloured labels (for example green for Available, amber for Reserved, blue for Borrowed, red for Overdue, grey for Retired), always together with the status text so that colour is not the only signal.



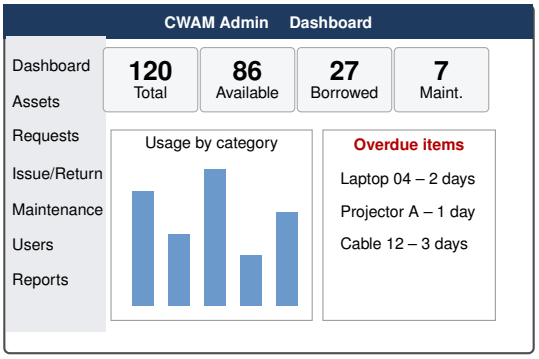
(a) Login screen



(b) Asset catalogue with search and filters



(c) AI Asset Assistant chat



(d) Administrator dashboard

Figure 3.1. User interface layouts of the main screens

- **Feedback:** a loading indicator is shown for any action that takes longer than one second, including AI responses.

3.1.2 Hardware Interfaces

- No special hardware is required, and the system does not communicate with any hardware device directly.
- It runs on standard computers, laptops, tablets and smartphones through the web browser, which handles the keyboard, mouse, touch screen and display.
- Asset reports and asset tags may be printed on any standard printer through the browser's normal print function; no special printer library is used.

3.1.3 Software Interfaces

Interface with the operating system. The server application runs as a Node.js process on Ubuntu 22.04 LTS or Windows 10/11 and uses the operating system for four services:

1. **Network sockets:** it listens for HTTPS requests (port 443 behind the reverse proxy) and connects to MySQL (port 3306).
2. **File system:** uploaded asset images and log files are stored in dedicated folders.
3. **Environment variables:** the database credentials, the JWT secret and the AI API key are read at start-up.
4. **System clock:** used for due dates, overdue checks and time stamps.

MySQL runs as an operating-system service that starts automatically, and a process manager restarts the Node.js process if it stops. On the client side the system does not use the operating system directly: it runs entirely inside the web browser, so it works on any operating system with a supported browser.

The other software components with which CWAM exchanges data are listed in Table 3.2.

Table 3.2. Software interfaces

Component	Name and version	Data exchanged and purpose
Operating System	Ubuntu 22.04 LTS or Windows 10/11	Sockets, file storage, environment variables and clock, as described above.
Database	MySQL 8.0 via the `mysql2` Node.js driver	SQL queries with parameters go in; result rows come out. Stores all users, assets, requests, borrowings, maintenance logs, notifications and chat logs.
Web framework	Node.js 18 LTS, Express 4	Receives HTTP requests from the client and returns JSON responses.
Client library	React 18 (built with Vite)	Renders the user interface and calls the REST API.
Authentication	`jsonwebtoken`, `bcrypt`	Creates and verifies JWTs; hashes and checks passwords.
AI Service	External LLM API (for example, the Claude API by Anthropic)	The user's question or need and the relevant asset data go out; the answer or ranked list comes back as JSON.
Reporting	Chart.js; a PDF and CSV export library	Draws dashboard charts and produces report files.

3.1.4 Communications Interfaces

- **Encryption:** all communication between the browser and the server, and between the server and the AI service, shall use HTTP over TLS 1.2 or later (HTTPS), so that all data, including passwords and tokens, is encrypted in transit.
- **Data format:** the client and server exchange data in JSON using the standard HTTP methods of a REST API (GET to read, POST to create, PUT or PATCH to update and DELETE to remove).
- **Authentication:** after login, the JWT is sent in the HTTP `Authorization: Bearer` header of every protected request.
- **Origin check:** the server accepts requests only from the system's own web address (CORS).

3.2 Functional Requirements

This section specifies the functions of the system, grouped into twelve modules. Each module states the user requirement and lists the system requirements with their priority.

3.2.1 User Registration and Login (FR-AU)

> **USER REQUIREMENT UR-AU**
> The user shall be able to create an account and log in to the system securely.

Table 3.3. System requirements: User Registration and Login

ID	Requirement	Priority
FR-AU-01	The system shall allow a new member to register with name, email, contact number and password.	**High**
FR-AU-02	The system shall reject registration if the email is already in use or the password is shorter than 8 characters.	**High**
FR-AU-03	The system shall store passwords as bcrypt hashes.	**High**

ID	Requirement (continued)	Priority
FR-AU-04	On successful login, the system shall issue a JWT that expires after a fixed period (for example, 24 hours).	High
FR-AU-05	The system shall show an error for an invalid email or password without revealing which one is wrong.	Medium
FR-AU-06	The system shall restrict each screen and API endpoint according to the user's role (Admin or Member).	High
FR-AU-07	The system shall allow the user to log out, which removes the token from the browser.	Medium
FR-AU-08	The system shall allow the admin to view all users and to deactivate or reactivate a member account; a deactivated user cannot log in.	Medium

3.2.2 Asset Management (FR-AM)

> **USER REQUIREMENT UR-AM**
> The administrator shall be able to add, view, update and delete assets.

Table 3.4. System requirements: Asset Management

ID	Requirement	Priority
FR-AM-01	The system shall allow the admin to add an asset with asset tag, name, category, location, condition, status, description and an optional image (JPG or PNG, up to 2 MB).	High
FR-AM-02	The asset tag shall be unique; the system shall reject a duplicate tag.	High
FR-AM-03	Condition shall be one of: New, Good, Fair, Damaged.	Medium
FR-AM-04	Status shall be one of: Available, Reserved, Borrowed, Under Maintenance, Retired.	High
FR-AM-05	The system shall allow the admin to edit any asset field except its usage history.	High
FR-AM-06	The system shall not allow deletion of an asset that is Reserved or Borrowed; such assets may only be marked Retired after return.	Medium
FR-AM-07	The system shall allow the admin to manage asset categories (add, rename, delete unused).	Low

3.2.3 Asset Search and Details (FR-AS)

> **USER REQUIREMENT UR-AS**
> The user shall be able to find assets and check their availability.

Table 3.5. System requirements: Asset Search and Details

ID	Requirement	Priority
FR-AS-01	The system shall provide a search box that matches asset name, tag and description.	High
FR-AS-02	The system shall provide filters for category, location, status and condition, which can be combined.	High
FR-AS-03	The system shall show results with name, image, category, location and status, 20 results per page.	Medium
FR-AS-04	Selecting an asset shall display its full details and upcoming reserved dates.	Medium
FR-AS-05	Retired assets shall be hidden from members.	Low

3.2.4 Asset Request (FR-RQ)

> **USER REQUIREMENT UR-RQ**
> The user shall be able to request an available asset for a period of time.

Table 3.6. System requirements: Asset Request

ID	Requirement	Priority
FR-RQ-01	The system shall show a *Request* button only on assets with status Available.	High
FR-RQ-02	The request form shall take purpose, start date and duration (1 to 30 days).	High
FR-RQ-03	The start date shall not be in the past.	Medium
FR-RQ-04	The system shall save the request with status Pending and notify the administrator.	High
FR-RQ-05	The system shall allow the user to cancel a request while it is Pending.	Medium
FR-RQ-06	The system shall show the user the status of every request: Pending, Approved, Rejected, Cancelled or Completed.	High

3.2.5 Request Approval (FR-AP)

> **USER REQUIREMENT UR-AP**
> The administrator shall be able to approve or reject asset requests.

Table 3.7. System requirements: Request Approval

ID	Requirement	Priority
FR-AP-01	The system shall list all pending requests with user, asset, purpose and dates.	High
FR-AP-02	The system shall not allow approval if the asset already has an approved request or borrowing for overlapping dates.	High
FR-AP-03	On approval, the system shall set the request to Approved and the asset to Reserved.	High
FR-AP-04	On rejection, the admin shall enter a reason and the system shall set the request to Rejected.	Medium
FR-AP-05	The system shall notify the user of the approval or rejection.	Medium
FR-AP-06	The admin shall be able to cancel an approved request whose asset has not been collected; the request is set to Cancelled and the asset back to Available.	Low

3.2.6 Borrowing and Returns (FR-BR)

> **USER REQUIREMENT UR-BR**
> The administrator shall be able to record when an asset is handed over and when it is returned.

Table 3.8. System requirements: Borrowing and Returns

ID	Requirement	Priority
FR-BR-01	The system shall allow the admin to issue an approved asset, creating a borrow record with issue date and due date, and setting the asset status to Borrowed.	High
FR-BR-02	The system shall allow the admin to record a return with return date and condition on return.	High
FR-BR-03	On return, the system shall set the asset to Available, or to Under Maintenance if the return condition is Damaged.	High
FR-BR-04	The system shall mark a borrow record as Overdue when the due date has passed without a return.	Medium
FR-BR-05	The system shall show overdue items on the admin dashboard and notify the borrower.	Medium
FR-BR-06	The system shall mark the related request as Completed after the return.	Medium

3.2.7 Maintenance Management (FR-MT)

> **USER REQUIREMENT UR-MT**
> The administrator shall be able to track repairs and maintenance of assets.

Table 3.9. System requirements: Maintenance Management

ID	Requirement	Priority
FR-MT-01	The system shall allow the admin to create a maintenance record with asset, issue description and date reported.	High
FR-MT-02	Creating a maintenance record shall set the asset status to Under Maintenance.	High
FR-MT-03	The admin shall be able to update the record with notes and cost.	Medium
FR-MT-04	Closing a maintenance record shall record the resolved date and set the asset to Available, or Retired if it cannot be repaired.	High
FR-MT-05	Members shall be able to report a problem with an asset they are borrowing, which creates a maintenance record for the admin to review.	Low

3.2.8 Usage History (FR-UH)

> **USER REQUIREMENT UR-UH**
> Users shall be able to see past usage of assets.

Table 3.10. System requirements: Usage History

ID	Requirement	Priority
FR-UH-01	The system shall show, for each asset, all past borrowings and maintenance records (admin only).	Medium
FR-UH-02	The system shall show each member their own request and borrowing history.	Medium
FR-UH-03	History records shall not be editable or deletable.	Medium

3.2.9 AI Asset Assistant (FR-AI)

USER REQUIREMENT UR-AI
The user shall be able to ask questions about assets in plain language and receive accurate answers.

Table 3.11. System requirements: AI Asset Assistant

ID	Requirement	Priority
FR-AI-01	The system shall provide a chat panel where the logged-in user types a question in natural language (up to 500 characters).	High
FR-AI-02	The server shall send the question to the LLM API together with a set of permitted data-lookup functions (for example, search assets, check availability, get my requests).	High
FR-AI-03	The server shall run the requested lookups on the database and return the results to the LLM to form the answer.	High
FR-AI-04	The assistant shall answer only from system data. If no matching asset exists, it shall say so and shall not invent one.	High
FR-AI-05	The assistant shall respect the user's role: a member shall not receive other members' personal data.	High
FR-AI-06	The assistant shall only read data; it shall not approve, delete or modify records. It may offer a link to the request form.	High
FR-AI-07	The system shall show a friendly error if the AI service is unavailable.	Medium
FR-AI-08	The system should store questions and answers in a chat log for review and improvement.	Low
FR-AI-09	The system should limit each user to 50 AI requests (Assistant and Recommendation together) per day, to control the cost of the LLM API.	Low

3.2.10 AI Asset Recommendation (FR-RC)

USER REQUIREMENT UR-RC
The user shall be able to describe their need and get suggestions of suitable assets.

Table 3.12. System requirements: AI Asset Recommendation

ID	Requirement	Priority
FR-RC-01	The system shall provide a form for requirement (free text, for example "laptop for video editing"), purpose, start date and duration.	High
FR-RC-02	The server shall first select only assets that are Available for the requested dates.	High
FR-RC-03	The server shall send the user's need and the candidate assets to the LLM, which ranks them and gives a one-line reason for each.	High
FR-RC-04	The system shall display up to five recommended assets in ranked order with their reasons.	Medium
FR-RC-05	Each recommended asset shall have a *Request* button that opens the request form pre-filled with the dates.	Medium
FR-RC-06	If no suitable asset is available, the system shall say so and suggest the nearest available date if one exists.	Low
FR-RC-07	If the AI service is unavailable, the system shall fall back to a simple rule-based ranking by category and keyword match.	Low

3.2.11 Dashboard and Reports (FR-RP)

> **USER REQUIREMENT UR-RP**
> The administrator shall be able to view the overall state and usage of assets.

Table 3.13. System requirements: Dashboard and Reports

ID	Requirement	Priority
FR-RP-01	The admin dashboard shall show the number of assets in each status.	High
FR-RP-02	The dashboard shall list overdue items and pending requests.	High
FR-RP-03	The system shall show charts of most-used assets and usage by category over a selected period.	Medium
FR-RP-04	The system shall show a maintenance summary (open records, total cost in a period).	Medium
FR-RP-05	The admin should be able to export reports as PDF or CSV.	Low

3.2.12 Notifications (FR-NT)

> **USER REQUIREMENT UR-NT**
> Users shall be informed of important events.

Table 3.14. System requirements: Notifications

ID	Requirement	Priority
FR-NT-01	The system shall create in-app notifications for request approval, rejection, an upcoming due date (one day before) and overdue items.	Medium
FR-NT-02	The system shall notify the admin of new requests and member problem reports.	Medium
FR-NT-03	Users shall be able to mark notifications as read.	Low

3.3 Behaviour Requirements

3.3.1 Use Case View

The use case diagram in Figure 3.2 covers the entire system and all of its actors, and each use case is described in one line below the diagram. Detailed use case descriptions (flows of events) are not included here, as required by the SRS template; they will be written for the Test Plan, where each one becomes the basis of test cases. The actors are:

- **Member** (primary actor): a registered user of the co-working space who searches for, requests and borrows assets and uses the AI features.
- **Administrator** (primary actor): a staff member who manages assets, users, requests, the issue and return of assets, maintenance and reports. The administrator can also log in and use the AI Assistant.
- **AI Service** (secondary actor): the external LLM API that the system calls to answer questions and rank assets. It never starts a use case and never accesses the database directly.

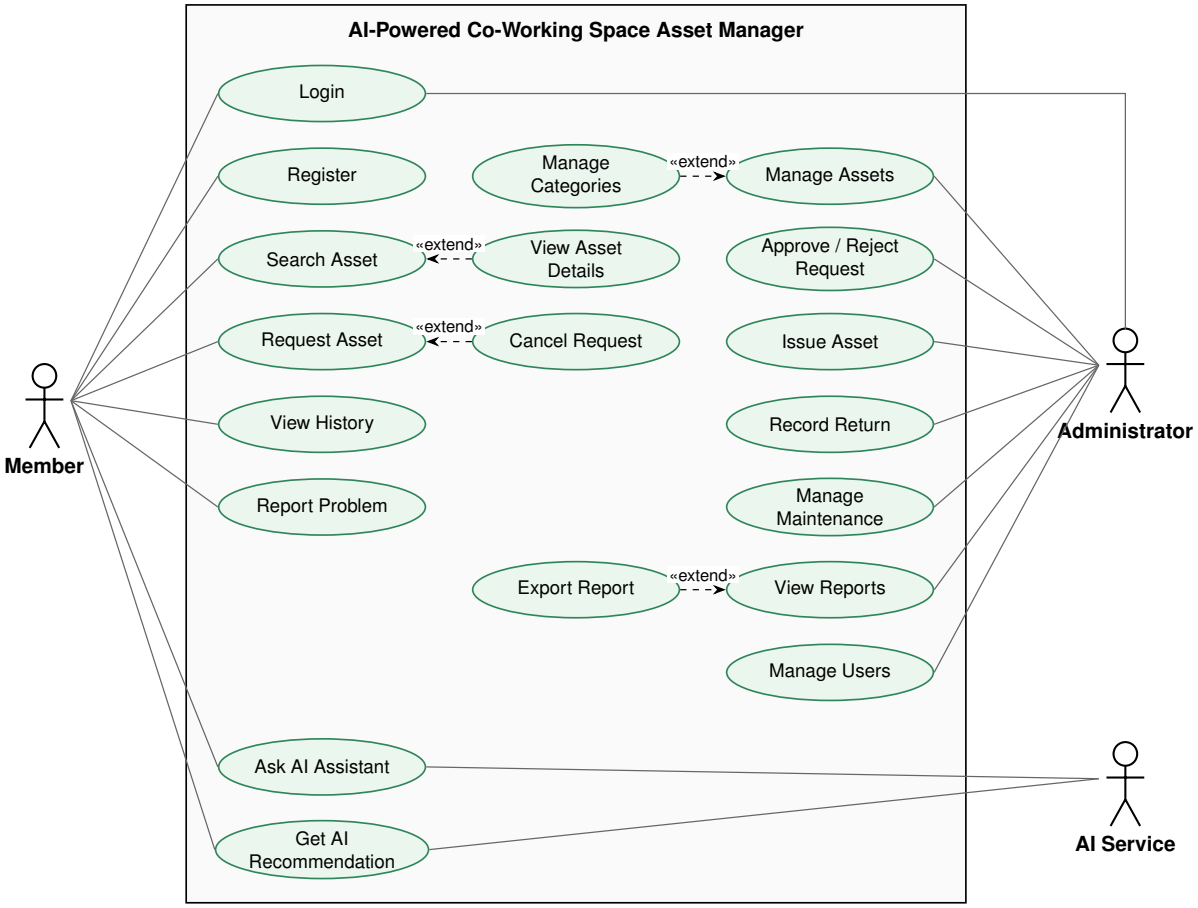


Figure 3.2. Use case diagram of CWAM

Use case descriptions

- **UC-01** **Login** *(Member, Admin)*: signs in with email and password before using any other function.
- **UC-02** **Register** *(Member)*: creates a new member account.
- **UC-03** **Search Asset** *(Member)*: searches and filters the asset catalogue.
- **UC-04** **View Asset Details** *(Member)*: views full details and availability; extends UC-03.
- **UC-05** **Request Asset** *(Member)*: submits a request for an available asset.
- **UC-06** **Cancel Request** *(Member)*: withdraws a pending request; extends UC-05.
- **UC-07** **View History** *(Member)*: views own requests and borrowing history.
- **UC-08** **Report Problem** *(Member)*: reports a fault with a borrowed asset.
- **UC-09** **Ask AI Assistant** *(Member, AI Service)*: asks a question in natural language and receives an answer from live data.
- **UC-10** **Get AI Recommendation** *(Member, AI Service)*: describes a need and receives ranked asset suggestions.
- **UC-11** **Manage Assets** *(Admin)*: adds, edits, retires or deletes assets.
- **UC-12** **Manage Categories** *(Admin)*: maintains asset categories; extends UC-11.
- **UC-13** **Approve / Reject Request** *(Admin)*: reviews pending requests and decides on them.
- **UC-14** **Issue Asset** *(Admin)*: records the hand-over of an approved asset.
- **UC-15** **Record Return** *(Admin)*: records the return and condition of a borrowed asset.
- **UC-16** **Manage Maintenance** *(Admin)*: logs, updates and closes maintenance records.
- **UC-17** **View Reports** *(Admin)*: views the dashboard and reports.
- **UC-18** **Export Report** *(Admin)*: downloads a report as PDF or CSV; extends UC-17.
- **UC-19** **Manage Users** *(Admin)*: views, activates or deactivates user accounts.

3.3.2 State Diagrams

The use cases above change the state of two key objects: the asset and the request. Their states and the events that move them from one state to another are shown in Figures 3.3 and 3.4. The same states are listed in the data dictionary (Appendix A). The system shall not allow any transition that is not shown in these diagrams.

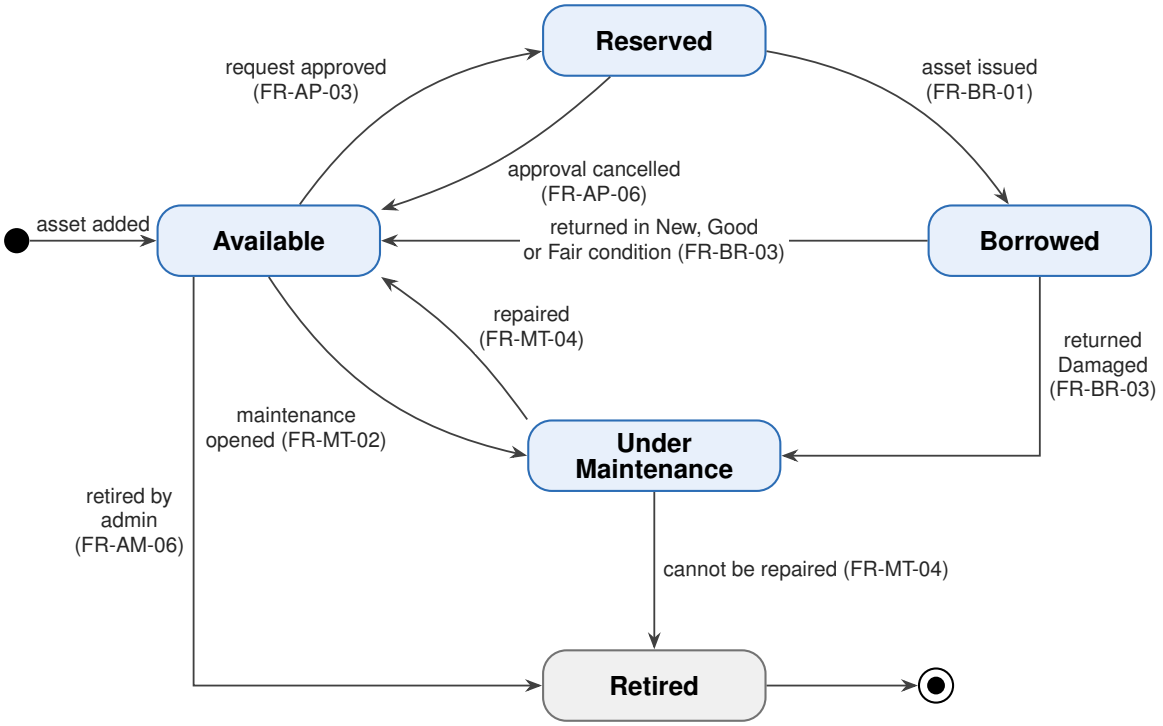


Figure 3.3. State diagram of an asset

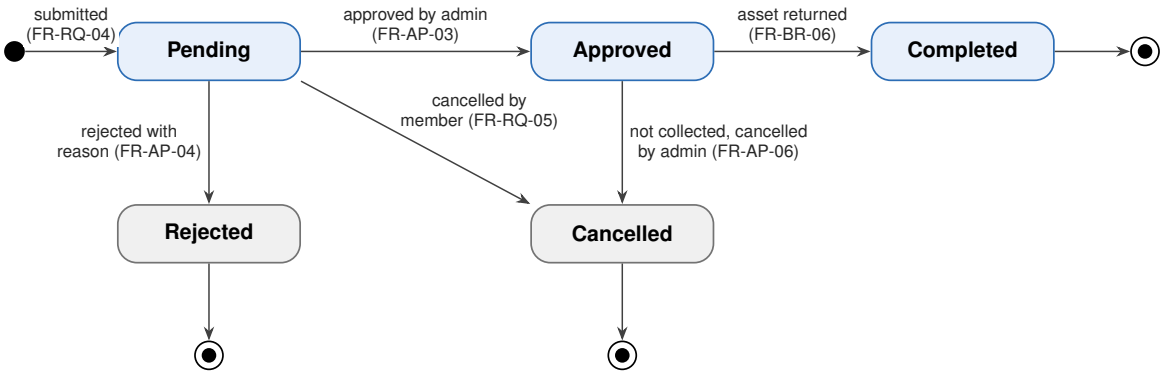


Figure 3.4. State diagram of an asset request

CHAPTER 4

Other Non-Functional Requirements

This chapter specifies how well the system must perform its functions: how fast it must respond, how it protects people and data from harm, and the quality attributes that the design must achieve.

4.1 Performance Requirements

The performance requirements below assume the sizing in Table 2.4 and the minimum server in Table 2.2. Each requirement gives its rationale so that developers understand its intent.

Table 4.1. Performance requirements

ID	Requirement	Rationale	Priority
NFR-PF-01	Each page of the application shall load within 3 seconds on a 10 Mbps connection.	Users abandon pages that feel slow; 3 seconds is a common limit for web usability.	**Medium**
NFR-PF-02	Search and filter results shall appear within 2 seconds for an inventory of up to 5,000 assets.	Search is the most frequent action; indexes on name, tag, status and category make this achievable.	**High**
NFR-PF-03	Approve, reject, issue and return actions shall be completed and shown on screen within 2 seconds.	Administrators perform these at the front desk while the member is waiting.	**Medium**
NFR-PF-04	AI Assistant answers and AI Recommendations should be displayed within 10 seconds, with a loading indicator while waiting.	The answer depends on an external LLM API; 10 seconds is acceptable for a chat reply when progress is shown.	**Medium**
NFR-PF-05	The system shall support at least 50 concurrent users while still meeting NFR-PF-01 to NFR-PF-03.	50 is the assumed maximum number of people using the system at the same time.	**Medium**
NFR-PF-06	The dashboard and any report covering up to 12 months of data shall be generated within 5 seconds.	Reports aggregate all borrow and maintenance records and are the heaviest database queries.	**Low**
NFR-PF-07	The overdue check shall run at least once every hour and finish within 30 seconds.	Overdue status and reminders must be up to date without slowing down normal use.	**Low**

4.2 Safety and Security Requirements

4.2.1 Safety Requirements

CWAM does not control any machinery, so it cannot cause physical harm directly. The possible harms are:

- Loss or corruption of records.
- Wrong decisions based on wrong data.
- Faulty equipment being lent out again.

The following safety requirements guard against them.

Table 4.2. Safety requirements

ID	Requirement	Priority
NFR-SF-01	The database shall be backed up automatically every 24 hours, keeping the last 7 backups. Restoring a backup shall be tested before the system is released.	High
NFR-SF-02	Approval, issue, return and closing of maintenance shall each run as a single database transaction. If any step fails, all its changes shall be rolled back, so that no asset or request is left in an inconsistent state.	High
NFR-SF-03	An asset returned as Damaged or reported faulty shall be set to Under Maintenance and shall not be lendable until an administrator closes its maintenance record, so that unsafe equipment (for example a damaged charger or cable) is not given to another member.	High
NFR-SF-04	Actions that cannot be undone (delete asset, retire asset, reject request, deactivate user) shall ask for confirmation before they are carried out.	Medium
NFR-SF-05	Usage history, borrow records and maintenance records shall never be physically deleted. Retired assets and deactivated users shall be kept together with their history.	Medium
NFR-SF-06	The AI features shall only have read access through the permitted lookup functions and shall not be able to create, change or delete any data.	High

4.2.2 Security Requirements

Expected level of security. The client expects a *moderate* level of security, suitable for an internal business application:

- **Data held:** personal data (names, e-mail addresses and contact numbers) and the asset records of the space, but no payment or financial data.
- **Main threats:** unauthorized use of administrator functions, members seeing other members' data, theft of passwords and the common web attacks listed in the OWASP Top Ten [5].
- **Required protection:** the system must authenticate every user, authorize every request by role and by ownership of the data, and protect data in transit and in storage.

The major security requirements are:

Table 4.3. Security requirements

ID	Requirement	Priority
NFR-SC-01	All users shall log in before using any function other than registration and login.	High
NFR-SC-02	Passwords shall be stored as bcrypt hashes with a cost factor of at least 10, never in plain text, and shall never be returned by the API.	High
NFR-SC-03	JWTs shall expire after 24 hours, and every protected API request shall verify the token and the user's role.	High
NFR-SC-04	A member shall only be able to view and change their own requests, borrowings, notifications and chat history; this shall be checked on the server for every request.	High
NFR-SC-05	All database queries shall use parameterized statements to prevent SQL injection.	High
NFR-SC-06	All user input shall be validated on both client and server, and all output shall be escaped to prevent cross-site scripting (XSS).	High
NFR-SC-07	In deployment, all traffic shall use HTTPS with TLS 1.2 or later.	High
NFR-SC-08	The AI API key, database credentials and JWT secret shall be kept in server environment variables only, and never sent to the browser or stored in the code repository.	High

ID	Requirement (continued)	Priority
NFR-SC-09	Only the minimum data needed shall be sent to the AI service; passwords and other members' personal details shall never be sent.	High
NFR-SC-10	After 5 failed login attempts within 15 minutes, further attempts for that account shall be blocked for 15 minutes.	Medium
NFR-SC-11	Administrator actions (approve, reject, issue, return, delete, deactivate) shall be recorded in an audit log with the administrator's ID and the time.	Medium

4.3 Software Quality Attributes

This section states the quality attributes that matter most to the client and the developers, how each will be achieved in the design, and how it will be checked. Their relative priority is:

1. **Correctness, reliability and security** come first, because wrong asset records or leaked personal data would make the system worse than the paper register it replaces.
2. **Usability and AI accuracy** come next, because members will only use the system if it is easier than asking staff, and will only trust the AI features if they are always right about what exists.
3. **Maintainability and design for change** follow, because the system will be extended after the course by other people.
4. The remaining attributes (availability, portability, interoperability, reusability, robustness, testability and scalability) are required but have lower priority.

Each subsection below states the requirement in a shaded box, followed by how it will be achieved in the design.

Ease of use versus ease of learning.

- For members, who use the system occasionally and without training, *ease of learning* is preferred over speed of use.
- For administrators, who use it every day, *ease of use* (few clicks, fast lists and filters) is preferred.

Table 4.4. Software quality attributes at a glance

ID	Attribute	Measurable target	Priority
QA-01	**Correctness**	No overlapping approvals; status always matches the state diagrams.	High
QA-02	**Reliability**	No loss of committed data; at most one failure needing a restart per month.	High
QA-03	**Availability**	At least 99% uptime per month; non-AI features work when the AI service is down.	Medium
QA-04	**Usability**	A new member finds and requests an asset within 3 minutes without help.	High
QA-05	**Maintainability**	A new developer sets up the system from the README within 1 hour.	Medium
QA-06	**Design for Change**	Each likely change is confined to one module or one configuration file.	Medium
QA-07	**Portability**	Server runs on Windows and Linux with no code changes.	Low
QA-08	**Interoperability**	All data available through a documented JSON API and CSV export.	Low
QA-09	**Reusability**	Authentication, catalogue and AI modules reusable without changes.	Low

ID	Attribute	Measurable target (continued)	Priority
QA-10	**Robustness**	No crash or corrupted data from invalid input or AI failure.	Medium
QA-11	**Testability**	Every FR covered by a test case; at least 70% coverage of server logic.	Medium
QA-12	**Scalability**	Four times the assumed load handled by hardware upgrades only.	Low
QA-13	**AI Accuracy**	100% of AI answers in the test set mention only existing, available assets.	High

4.3.1 Correctness

> **QA-01** The status of every asset and request shall always reflect the real situation. An asset can never be approved for two overlapping periods, and can never be in two states at once.

- Status changes happen only through the transitions in Figures 3.3 and 3.4, implemented in one server module; no other code may change a status directly.
- The overlap check and the approval run in the same database transaction, with the asset row locked (`SELECT ... FOR UPDATE`) so that two administrators cannot approve conflicting requests at the same moment.
- Database constraints back up the code: unique asset tags, foreign keys, and ENUM types that accept only the valid states.

Verification: Unit tests for every allowed and every forbidden transition, and a test in which two overlapping requests are approved at the same time.

4.3.2 Reliability

> **QA-02** The system shall operate without failure during normal use, with no loss of committed data and no more than one failure per month that needs a restart.

- Database transactions for every multi-step operation (NFR-SF-02) and daily backups (NFR-SF-01).
- A central error handler in Express, so that an error in one request returns an error message instead of stopping the server.
- A process manager (PM2) that restarts the server automatically, and error logs with time and request details for diagnosis.

Verification: A 24-hour test with simulated users, and fault tests in which the database or the AI service is stopped during a request.

4.3.3 Availability

> **QA-03** The system shall be available 24 hours a day with at least 99% uptime per month (no more than about 7 hours of unplanned downtime), excluding planned maintenance announced in advance. If the AI service is unavailable, all other features shall remain available.

- Automatic restart after failures and planned maintenance at night when the space is closed.
- The AI integration is isolated in its own module with a 10-second timeout and a fallback, so its failure cannot block other features.
- A health-check endpoint (`/api/health`) that can be monitored by a free uptime service.

4.3.4 Usability

> **QA-04** A new member shall be able to find and request a suitable asset within 3 minutes on first use, without help. An administrator shall be able to approve a pending request from the dashboard in no more than 3 clicks.

- A consistent layout and the GUI standards of Section 3.1.1: the same navigation, buttons and message style on every screen.
- Plain-language labels and error messages that explain how to fix the problem; example questions in the AI chat panel.
- The AI Assistant and AI Recommendation give members a second, conversational way to find assets.
- A responsive layout for phones and a usability test with at least five students before release.

4.3.5 Maintainability

> **QA-05** A developer who is new to the project shall be able to install and run the system from the README within 1 hour, and a typical change or bug fix shall affect only one module.

- A layered server (routes, controllers, services, data access) and a component-based React client, so that each part has one clear responsibility.
- The coding standards of constraint C-08 (ESLint and Prettier), meaningful names, comments for non-obvious logic and an API reference.
- Version control with Git, small commits and a branch for each feature; configuration in a `.env` file instead of in the code.

4.3.6 Design for Change

> **QA-06** Changes that are likely in the future shall be possible by changing one module or one configuration file, without changes spreading through the rest of the system.

The likely changes have been identified in advance, and each is hidden behind a single module (information hiding):

- **A different AI provider:** all AI calls go through one AI service module with two functions, *ask assistant* and *rank assets*. Changing provider means rewriting only this module.
- **New categories, locations or asset types:** these are data in the database, not code, and are managed from the user interface.
- **Different business limits** (for example the maximum borrowing period or the AI daily limit): all limits are named constants in one configuration file (Appendix A, Table A.1).
- **New notification channels** such as e-mail or SMS: notifications are sent through one notification service; in-app messages are its first channel, and new channels can be added without changing the modules that create notifications.
- **A mobile app or another client:** all functions are exposed through the REST API, which does not depend on the React client.
- **Several co-working sites:** all database access goes through one data-access layer, so adding a site field affects that layer and the screens that show locations.

4.3.7 Portability

> **QA-07** The server shall run on Windows 10/11 and Ubuntu 22.04 LTS without code changes, and the client shall run in any supported browser on any operating system.

- Only cross-platform technologies are used (Node.js, MySQL, React), and settings come from environment variables.
- File paths are built with the Node.js `path` module; no operating-system-specific commands are used.
- The system will be developed on Windows and tested on Ubuntu before release.

4.3.8 Interoperability

> **QA-08** Other software shall be able to exchange data with CWAM through a documented REST API in JSON and through CSV report files.

- Standard formats only: JSON with ISO 8601 dates in the API, and CSV for exports (FR-RP-05).
- An API reference (for example a Postman collection) lists every endpoint with its inputs and outputs.

4.3.9 Reusability

> **QA-09** The authentication, asset catalogue and AI integration modules shall be reusable in similar systems, such as a university lab or library equipment manager, without changes to their code.

- These modules contain no co-working-specific rules, and authentication and role checks are Express middleware.
- Common interface parts (data table, form, status label, chat panel) are written as generic React components with parameters.

4.3.10 Robustness

> **QA-10** Invalid input, lost connections and AI service failures shall never crash the system or corrupt data; the user shall always receive a clear message.

- Validation of every input on client and server with a schema-validation library, and a central error handler.
- A timeout on every AI call, with a user-friendly error or the rule-based fallback (FR-AI-07, FR-RC-07).
- Transactions that roll back on failure.

4.3.11 Testability

> **QA-11** Every functional requirement shall be verifiable by at least one test case, and automated tests shall cover at least 70% of the server's business logic.

- Every requirement has a unique ID, and Appendix B links use cases to requirements.
- Business logic is kept in service modules separate from the Express routes, so it can be unit-tested with Jest.
- The API is tested with Supertest and Postman against a test database with seed data; the AI service is replaced by a mock in automated tests so that results are repeatable.

4.3.12 Scalability

> **QA-12** The system shall handle four times the assumed load (20,000 assets and 200 concurrent users) by upgrading hardware or configuration only, without redesign.

- The server is stateless (login state is in the JWT, not in server memory), so more server instances can be added behind a load balancer.
- Database tables are indexed, all lists are paginated, and images are stored as files rather than in the database.

4.3.13 AI Accuracy

> **QA-13** The AI features shall only mention assets that exist in the database, and recommendations shall only include assets that are available for the requested dates. In a test set of 30 questions, 100% of answers shall meet this rule.

- The LLM receives asset data only from the database, through the permitted lookup functions (FR-AI-02, FR-AI-03); it is instructed to say when nothing is found.
- Recommendation candidates are filtered by SQL before ranking (FR-RC-02), so the LLM can only choose from available assets.
- The server checks every asset ID in the AI response against the candidate list and removes any that are not in it before the answer is shown.

CHAPTER 5

Other Requirements

This chapter defines requirements that are not covered elsewhere in the SRS: database, legal and privacy, internationalization and reuse requirements, followed by enhancements planned for later versions.

5.1 Database Requirements

Table 5.1. Database requirements

ID	Requirement	Priority
OR-DB-01	The system shall use MySQL 8.0 with the InnoDB storage engine and the `utf8mb4` character set.	**High**
OR-DB-02	The schema shall be in at least third normal form, with primary keys, foreign keys and ENUM constraints for all status fields, as listed in Appendix A.	**High**
OR-DB-03	Indexes shall exist on the asset tag, asset status and category, request asset and dates, and borrow due date, to meet NFR-PF-02.	**Medium**
OR-DB-04	An asset or user that has requests, borrowings or maintenance records shall not be deleted; it shall be retired or deactivated instead.	**Medium**
OR-DB-05	Schema changes shall be made through versioned migration scripts kept in the repository, with a seed script that creates the first administrator account and sample data.	**Medium**
OR-DB-06	Borrow, maintenance and audit records shall be kept for at least 3 years; AI chat logs shall be deleted after 90 days.	**Low**

5.2 Legal and Privacy Requirements

Table 5.2. Legal and privacy requirements

ID	Requirement	Priority
OR-LG-01	The system shall collect only the personal data it needs: name, e-mail address and contact number.	**High**
OR-LG-02	The registration screen shall show a short privacy notice stating what data is stored, why, and that questions typed into the AI features are sent to an external AI provider.	**Medium**
OR-LG-03	On a member's request, the administrator shall be able to deactivate the account and remove the member's contact details, while keeping anonymous usage history.	**Low**
OR-LG-04	Only open-source libraries with licences that allow free use (such as MIT, Apache 2.0 or BSD) shall be used, and they shall be listed in the repository.	**Medium**
OR-LG-05	Use of the LLM API shall follow the provider's terms of service and usage policies.	**Medium**
OR-LG-06	The login page shall state that the system is for authorized users only; unauthorized access to an information system is an offence under Pakistan's Prevention of Electronic Crimes Act, 2016.	**Low**

5.3 Internationalization Requirements

Table 5.3. Internationalization requirements

ID	Requirement	Priority
OR-IN-01	The interface of Version 1.0 shall be in English, with all interface text kept in a separate language file so that Urdu can be added later without code changes.	Low
OR-IN-02	Dates shall be displayed as DD/MM/YYYY and times in Pakistan Standard Time (UTC+5); time stamps shall be stored in UTC.	Medium
OR-IN-03	Maintenance costs shall be recorded in Pakistani Rupees (PKR) with two decimal places.	Low

5.4 Reuse Objectives

Table 5.4. Reuse objectives

ID	Requirement	Priority
OR-RU-01	The project shall reuse well-established open-source components instead of writing its own: React, Express, `mysql2`, `jsonwebtoken`, `bcrypt` and Chart.js.	Medium
OR-RU-02	The authentication middleware and the generic interface components shall be written so that they can be reused in later versions and in similar projects (QA-09).	Low
OR-RU-03	The AI integration module shall be usable by any future feature that needs an LLM with data-lookup functions.	Low

5.5 Future Enhancements

The following features are outside the scope of Version 1.0 but have been considered in the design, so that they can be added later:

- **QR code labels and scanning:** print a QR label for each asset and scan it with a phone camera to speed up issue and return.
- **E-mail and SMS notifications** for approvals, due dates and overdue items, added as new channels of the notification service.
- **Desk and meeting-room booking** using the same request and approval workflow.
- **Multi-site support** for co-working chains with several branches.
- **Predictive maintenance:** the AI analyses usage and fault history and suggests which assets need servicing before they fail.
- **Mobile application** for Android and iOS using the same REST API.
- **Urdu interface** using the separate language file (OR-IN-01).
- **Integration with membership and billing,** for example to charge late fees for overdue items.

APPENDIX A

Data Dictionary

This data dictionary tracks the constants, state variables, inputs and outputs used in this document, with their description and the operations and requirements they relate to. Section A.5 lists the database tables that store the data.

A.1 Constants

Table A.1. Constants

Constant	Value	Description	Requirements
MIN_PASSWORD_LENGTH	8 characters	Shortest password accepted at registration.	FR-AU-02
BCRYPT_COST	10	Work factor used when hashing passwords.	FR-AU-03, NFR-SC-02
JWT_EXPIRY	24 hours	Lifetime of a login token.	FR-AU-04, NFR-SC-03
MAX_LOGIN_ATTEMPTS	5 in 15 minutes	Failed logins allowed before the account is blocked.	NFR-SC-10
LOCKOUT_PERIOD	15 minutes	Time an account stays blocked after too many failed logins.	NFR-SC-10
MAX_IMAGE_SIZE	2 MB (JPG, PNG)	Largest asset image accepted.	FR-AM-01
PAGE_SIZE	20	Number of assets on one page of search results.	FR-AS-03
MIN_BORROW_DAYS	1 day	Shortest request duration.	FR-RQ-02
MAX_BORROW_DAYS	30 days	Longest request duration.	FR-RQ-02
DUE_REMINDER_LEAD	1 day	How long before the due date a reminder is sent.	FR-NT-01
OVERDUE_CHECK_INTERVAL	1 hour	How often the system looks for overdue borrowings.	FR-BR-04, NFR-PF-07
NOTIFICATION_POLL	60 seconds	How often the client checks for new notifications.	FR-NT-01
MAX_QUESTION_LENGTH	500 characters	Longest question accepted by the AI Assistant.	FR-AI-01
AI_DAILY_LIMIT	50 per user per day	Maximum number of AI requests per user.	FR-AI-09
AI_TIMEOUT	10 seconds	Time after which an AI call is abandoned.	NFR-PF-04, FR-AI-07, FR-RC-07
MAX_RECOMMENDATIONS	5	Number of assets shown by the AI Recommendation.	FR-RC-04
BACKUP_INTERVAL	24 hours	Time between automatic database backups.	NFR-SF-01

Constant	Value	Description (continued)	Requirements
BACKUP_RETENTION	7 backups	Number of backups kept.	NFR-SF-01
CHAT_LOG_RETENTION	90 days	How long AI chat logs are kept.	FR-AI-08, OR-DB-06

A.2 State Variables

Table A.2. State variables and their possible states

Variable	Possible states	Description and changing operations	Requirements
asset.status	Available, Reserved, Borrowed, Under Maintenance, Retired	Availability of an asset (Figure 3.3); changed by approval, issue, return, maintenance and retirement.	FR-AM-04, FR-AM-06, FR-AP-03, FR-AP-06, FR-BR-01, FR-BR-03, FR-MT-02, FR-MT-04
asset.condition	New, Good, Fair, Damaged	Physical condition; set when an asset is added, updated or returned.	FR-AM-03, FR-BR-02
request.status	Pending, Approved, Rejected, Cancelled, Completed	Progress of a request (Figure 3.4); changed by request, cancel, decision and return.	FR-RQ-04, FR-RQ-05, FR-RQ-06, FR-AP-03, FR-AP-04, FR-AP-06, FR-BR-06
borrow.status	Active, Overdue, Returned	State of a borrowing: Active when issued, Overdue after the due date, Returned on return.	FR-BR-01, FR-BR-02, FR-BR-04
maintenance.status	Open, In Progress, Closed	Progress of a repair: opened by an admin or a problem report, closed when resolved.	FR-MT-01, FR-MT-03, FR-MT-04, FR-MT-05
maintenance.outcome	Repaired, Retired	Result recorded when a repair is closed.	FR-MT-04
user.role	Admin, Member	Access level; members self-register, the first admin is seeded.	FR-AU-01, FR-AU-06
user.account_status	Active, Deactivated	Whether the user may log in; changed by Manage Users.	FR-AU-08
notification.is_read	Unread, Read	Whether a notification has been seen.	FR-NT-03
session	Logged out, Logged in	Whether the browser holds a valid JWT.	FR-AU-04, FR-AU-07

A.3 Inputs

Table A.3. System inputs

Input	Type / format	Validation	Requirements
Name	Text	Required, 2 to 100 characters.	FR-AU-01
E-mail	Text (e-mail)	Valid format; unique at registration.	FR-AU-01, FR-AU-02

Input	Type / format	Validation (continued)	Requirements
Contact number	Text	10 to 15 digits, optional leading +.	FR-AU-01
Password	Text (hidden)	At least 8 characters.	FR-AU-02
Asset tag	Text	Required, unique, up to 30 characters.	FR-AM-01, FR-AM-02
Asset name, category, location, description	Text and list	Name, category and location required.	FR-AM-01
Condition	List	One of the asset.condition values.	FR-AM-03, FR-BR-02
Asset image	File	JPG or PNG, up to 2 MB.	FR-AM-01
Search text and filters	Text and lists	Up to 100 characters; filters from fixed lists.	FR-AS-01, FR-AS-02
Purpose	Text	Required, up to 255 characters.	FR-RQ-02
Start date	Date	Today or later.	FR-RQ-02, FR-RQ-03, FR-RC-01
Duration	Whole number of days	1 to 30 days.	FR-RQ-02, FR-RC-01
Rejection reason	Text	Required when rejecting.	FR-AP-04
Return date and condition	Date and list	Return date not before issue date.	FR-BR-02
Issue description, notes, cost	Text and decimal	Description required; cost 0 or more.	FR-MT-01, FR-MT-03
AI question	Text	1 to 500 characters.	FR-AI-01
Requirement (need)	Text	Required, up to 500 characters.	FR-RC-01
Report type and date range	List and dates	End date not before start date.	FR-RP-03, FR-RP-05

A.4 Outputs

Table A.4. System outputs

Output	Description	Shown on / sent to	Requirements
JWT	Signed token containing the user ID and role.	Browser, sent with each request	FR-AU-04
Asset list	Matching assets with name, image, category, location and status, one page at a time.	Catalogue	FR-AS-03
Asset details	Full details of one asset and its reserved dates.	Asset Details	FR-AS-04
Request status	Current status of each of the member's requests.	My Requests	FR-RQ-06
Borrow record	Issue date, due date, return date and condition.	Issue/Return, history	FR-BR-01, FR-BR-02
Overdue list	Borrowings past their due date.	Admin Dashboard	FR-BR-05, FR-RP-02
Notifications	In-app messages about requests, due dates and problem reports.	Notification bell	FR-NT-01, FR-NT-02
AI answer	Plain-language reply based on live system data.	AI Assistant	FR-AI-03, FR-AI-04

Output	Description (continued)	Shown on / sent to	Requirements
Recommendation list	Up to 5 ranked assets with a reason for each.	AI Recommendation	FR-RC-03, FR-RC-04
Dashboard figures	Counts by status, usage charts and maintenance summary.	Admin Dashboard	FR-RP-01, FR-RP-03, FR-RP-04
Report file	Report in PDF or CSV format.	Downloaded file	FR-RP-05
Messages	Validation errors, confirmations and success messages.	All screens	NFR-SF-04, QA-04

A.5 Database Tables

The data is stored in the MySQL tables in Table A.5:

- Every primary key (PK) is an auto-increment `INT`; FK marks a foreign key.
- Status and condition fields are ENUM types holding exactly the states in Table A.2.
- Dates use `DATE`, time stamps `DATETIME` in UTC, passwords a `CHAR(60)` bcrypt hash and costs `DECIMAL(10,2)`.

Table A.5. Database tables and fields

Table	Fields	Purpose
`users`	user_id (PK), name, email (unique), contact_no, password_hash, role, account_status, failed_logins, locked_until, created_at	Members and administrators.
`categories`	category_id (PK), name (unique), description	Asset categories.
`assets`	asset_id (PK), asset_tag (unique), name, category_id (FK), location, condition, status, description, image_path, created_at, updated_at	Inventory of shared assets.
`asset_requests`	request_id (PK), user_id (FK), asset_id (FK), purpose, start_date, end_date, status, rejection_reason, decided_by (FK users), decided_at, created_at	Requests and the administrator's decisions.
`borrow_records`	borrow_id (PK), request_id (FK, unique), asset_id (FK), user_id (FK), issued_at, due_date, returned_at, return_condition, status	Issue and return of assets.
`maintenance_logs`	maintenance_id (PK), asset_id (FK), reported_by (FK users), issue_description, notes, cost, status, outcome, reported_at, resolved_at	Repairs and their cost.
`notifications`	notification_id (PK), user_id (FK), type, message, is_read, created_at	In-app notifications.
`ai_chat_logs`	chat_id (PK), user_id (FK), feature, prompt, response, created_at	AI exchanges (kept 90 days).
`audit_logs`	log_id (PK), user_id (FK), action, entity, entity_id, created_at	Administrator actions (NFR-SC-11).

Relationships.

- **categories – assets:** one-to-many (each asset belongs to one category).
- **users – asset_requests** and **assets – asset_requests:** one-to-many (a member makes many requests; an asset receives many requests over time).
- **asset_requests – borrow_records:** one-to-zero-or-one (an approved request leads to at most one borrowing).
- **assets – maintenance_logs:** one-to-many (an asset can be repaired many times).
- **users – notifications**, **users – ai_chat_logs** and **users – audit_logs:** one-to-many.

APPENDIX B

Requirements Traceability Matrix

The matrix below links every use case in Section 3.3.1 to the functional requirements that implement it, so that each requirement can be traced through design and testing.

Table B.1. Requirements traceability matrix

ID	Use Case	Functional Requirements
UC-01	**Login**	FR-AU-04, FR-AU-05, FR-AU-06, FR-AU-07
UC-02	**Register**	FR-AU-01, FR-AU-02, FR-AU-03
UC-03	**Search Asset**	FR-AS-01, FR-AS-02, FR-AS-03, FR-AS-05
UC-04	**View Asset Details**	FR-AS-04
UC-05	**Request Asset**	FR-RQ-01, FR-RQ-02, FR-RQ-03, FR-RQ-04, FR-RQ-06
UC-06	**Cancel Request**	FR-RQ-05
UC-07	**View History**	FR-UH-02, FR-UH-03
UC-08	**Report Problem**	FR-MT-05, FR-NT-02
UC-09	**Ask AI Assistant**	FR-AI-01 to FR-AI-09
UC-10	**Get AI Recommendation**	FR-RC-01 to FR-RC-07, FR-AI-09
UC-11	**Manage Assets**	FR-AM-01 to FR-AM-06
UC-12	**Manage Categories**	FR-AM-07
UC-13	**Approve / Reject Request**	FR-AP-01 to FR-AP-06, FR-NT-01
UC-14	**Issue Asset**	FR-BR-01
UC-15	**Record Return**	FR-BR-02 to FR-BR-06
UC-16	**Manage Maintenance**	FR-MT-01 to FR-MT-04
UC-17	**View Reports**	FR-RP-01 to FR-RP-04, FR-UH-01
UC-18	**Export Report**	FR-RP-05
UC-19	**Manage Users**	FR-AU-06, FR-AU-08